Project RENDERING — Technical Report

Computer Graphics Class Presentation Guide

Hossein F. · Christian C. · Abraham V.

2026

1. Architecture Overview

The app has three layers working together:

Frontend UI (`src/App.tsx`) — Manages global state (`SceneSettings`), wires user events to state updates, and renders the header, controls panel, and 3D viewport side by side.

3D Renderer (`src/components/Scene.tsx`) — A React Three Fiber (`@react-three/fiber`) canvas that translates `SceneSettings` into live Three.js scene objects: lights, materials, geometry, and camera.

Python Logic Layer (`src/services/pythonEngine.ts` + `src/services/rendering_logic.py`) — Pyodide (Python compiled to WebAssembly) runs real Python in the browser to parse OBJ files and compute a reference shading value.

---

2. The Lighting Equation

Every material model in the app is a variant or subset of the **Phong Illumination Model**:

$$I = I_a k_a + I_l[k_d(N \cdot L) + k_s(N \cdot H)^n]$$

Symbol	Meaning
I	Final pixel intensity
I_a	Ambient light intensity (slider 0–2)
k_a	Ambient reflectance coefficient
I_l	Point light intensity (slider 0–10)
k_d	Diffuse reflectance coefficient (the object's base color)
k_s	Specular reflectance coefficient
N	Surface normal vector at the fragment
L	Unit vector from fragment toward the light source
V	Unit vector from fragment toward the camera
H	Halfway vector = normalize($L + V$)
n	Shininess exponent (controls specular highlight size)

---

3. Shading Models

The four models are a progression from no computation to full physics simulation, implemented via Three.js material classes.

3.1 Unlit / Basic (`meshBasicMaterial`)

$$I = k_d$$

No lighting equation is evaluated. Every pixel renders the raw albedo color regardless of light position or intensity. Use case: debugging geometry without lighting interference.

3.2 Lambert / Diffuse (`meshLambertMaterial`)

$$I = I_a k_a + I_l \cdot k_d \cdot \max(N \cdot L, 0)$$

Diffuse-only. $N \cdot L$ is the cosine of the angle between the surface normal and the light direction. When the surface faces the light directly (angle = 0°), $N \cdot L = 1 \rightarrow$ full brightness. When it faces away (angle ≥ 90°), $\max(N \cdot L, 0) = 0 \rightarrow$ no contribution. Roughness and Metalness are inactive because there is no specular term.

Three.js `meshLambertMaterial` uses **Gouraud shading**: the dot product is computed once per vertex and then linearly interpolated across each triangle face. This is fast but can miss specular highlights that fall between vertices.

3.3 Blinn-Phong (`meshPhongMaterial`)

$$I = I_a k_a + I_l[k_d(N \cdot L) + k_s(N \cdot H)^n]$$

Adds a **specular term**. Instead of reflecting the light ray and comparing to V (classic Phong), Blinn's optimization computes the halfway vector $H = \text{normalize}(L + V)$ and uses $(N \cdot H)^n$. This is cheaper and nearly identical visually.

- **Low roughness → high *n* → tight, bright highlight** (polished surface)
- **High roughness → low *n* → wide, dim highlight** (matte surface)

The app maps: $n = \max(1, \text{round}((1 - \text{roughness}) \times 300))$

Metalness is inactive here because Blinn-Phong has no Fresnel equation and no conductor/dielectric distinction.

Three.js `meshPhongMaterial` uses **Phong shading**: the full equation is evaluated per fragment (pixel), correctly capturing specular highlights even within a triangle.

3.4 PBR / GGX (`meshStandardMaterial`)

Physically Based Rendering models surfaces as a collection of microscopic mirrors called **microfacets**.

Roughness (α): Controls the statistical spread of microfacet orientations. Low $\alpha \rightarrow$ tight specular lobe. High $\alpha \rightarrow$ broad, scattered highlight.

Metalness (F_0): Controls the **Fresnel effect** — how much light is reflected vs. absorbed. Dielectrics (metalness = 0, e.g. plastic) reflect a small constant amount and have a diffuse layer. Conductors (metalness = 1, e.g. chrome) absorb diffuse energy — all non-specular light is absorbed, not scattered. This is why metals have no visible diffuse color.

The GGX (Trowbridge-Reitz) normal distribution function used internally:

$$D(H) = \frac{\alpha^2}{\pi \left[(N \cdot H)^2(\alpha^2 - 1) + 1\right]^2}$$

All controls (ambient, point intensity, roughness, metalness) are active in this mode.

---

4. Vectors in the Shading Pipeline

Understanding these four unit vectors is the key to understanding all shading:

```
N = surface normal at fragment (points away from surface)
L = normalize(lightPosition − fragmentPosition)
V = normalize(cameraPosition − fragmentPosition)
H = normalize(L + V)   ← Blinn's halfway vector
```

$N \cdot L$ (dot product) is the cosine of the angle between the surface and the light. It drives **diffuse** brightness.

$N \cdot H$ raised to the power n drives the **specular** highlight. The closer H is to N, the more the surface is oriented to perfectly reflect the light toward the camera.

---

5. Ambient Light

$$I_a k_a$$

Ambient light is a **constant additive term** — it adds the same brightness to every fragment regardless of orientation or distance from the light. It approximates global indirect illumination (light bouncing off walls, sky, etc.) without actually computing it. The

app defaults to `0.05` to keep the scene mostly dark, giving the other shading terms visual contrast.

---

6. Wireframe Mode

When wireframe is enabled, the app deliberately switches to `meshBasicMaterial` regardless of the selected shading model:

```
if (wireframe) return <meshBasicMaterial color={color} wireframe />;
```

Why unlit for wireframe? If you use a lit material (Phong, Lambert) in wireframe mode, the lighting shader runs on the edge geometry, not face geometry. Edge triangles have completely different $N \cdot L$ values than the solid faces, making the scene appear to change its entire lighting when wireframe is toggled. Using an unlit material keeps wireframe visually consistent and shows only mesh topology without confusing lighting artifacts.

---

7. Geometry

Four primitives are available, all generated procedurally by Three.js:

Shape	Three.js Class	Key Parameters
Cube	`BoxGeometry`	2×2×2 units
Sphere	`SphereGeometry`	radius 1.5, 32×32 segments
Plane	`PlaneGeometry`	3×3 units
Torus	`TorusGeometry`	major radius 1.2, tube 0.4, 16×100 segments

Segment count matters for shading: more segments = more vertices = finer $N \cdot L$ interpolation = smoother diffuse gradient. With Gouraud shading (Lambert), a low-poly sphere looks faceted; with Phong shading (per-fragment), even a coarser sphere can look

smooth.

---

8. OBJ File Loading and the Python Layer

When a `.obj` file is uploaded, it goes through a two-phase pipeline:

Phase 1 — Python parsing (`rendering_logic.py`):

```
class OBJParser:
    def parse(self, obj_string):
        # Parses 'v x y z' lines into vertex list
        # Parses 'f i j k' face lines, strips texture/normal indices
        # Fan-triangulates polygons:
        #   [0,1,2,3,4] → [0,1,2], [0,2,3], [0,3,4]
        for i in range(1, len(indices) - 1):
            self.faces.append([indices[0], indices[i], indices[i + 1]])
```

OBJ files can have n-gon faces (quads, pentagons, etc.). Fan triangulation converts them to triangles — the only primitive GPUs render natively — by anchoring on vertex 0 and fanning through the polygon.

Phase 2 — Three.js geometry (`Scene.tsx`):

```
const positions = new Float32Array(objData.vertices.flat());
geom.setAttribute('position', new THREE.BufferAttribute(positions, 3));
geom.setIndex(new THREE.BufferAttribute(indices, 1));
geom.computeVertexNormals();
```

`computeVertexNormals()` calculates normals by averaging face normals at shared vertices — this is what allows the shading to look smooth across the loaded mesh.

The geometry is also **normalized**: centered on the bounding box center and scaled so its largest dimension = 2 units, ensuring any OBJ model fits the scene.

Pyodide loads CPython compiled to WebAssembly and runs inside the browser tab — no server required. The `calculate_shading` function demonstrates how `color × intensity` maps to the $I_l \cdot k_d$ term in the illumination equation.

9. Camera System

The camera uses **perspective projection** with a 50° field of view, positioned at $[5, 5, 5]$ looking at the origin. Perspective projection divides x and y by z, making farther objects appear smaller — matching how a physical camera or eye works.

OrbitControls implements arcball-style camera navigation:

Input	Action
Left drag	Rotate (changes spherical coordinates θ, ϕ around target)
Right drag	Pan (translates both camera and target)
Scroll	Zoom (changes the radial distance)
`R` key	Reset camera to $[5, 5, 5]$
`Space`	Toggle auto-rotate
`W`	Toggle wireframe

`enableDamping` adds inertia so the camera coasts to a stop after input ends.

10. Rendering Pipeline Summary

```
OBJ/Primitive → BufferGeometry (vertices, indices, normals)
                        ↓
        Vertex Shader (transforms to clip space, passes N, L, V)
                        ↓
        Rasterization (fills triangles with fragments)
                        ↓
        Fragment Shader (evaluates Iₐkₐ + Iₗ[kd(N·L) + ks(N·H)ⁿ])
                        ↓
        Framebuffer → Canvas → Screen
```

Three.js handles the GLSL shaders internally but exposes the high-level parameters (color, roughness, metalness, shininess) through its material classes, which the Controls panel manipulates in real time.

---

11. Key Questions to Be Able to Answer

Why does the dark side of a sphere stay dark? $\max(N \cdot L, 0)$ clamps the dot product to zero when the surface faces away from the light, preventing negative (subtractive) contributions.

Why does metalness not apply to Blinn-Phong? Blinn-Phong has a fixed k_s coefficient. It doesn't model how the Fresnel equations change reflectance based on the angle of incidence, nor does it distinguish how dielectrics and conductors interact with light at the energy-transfer level.

Why does wireframe switch to unlit? Lit materials compute $N \cdot L$ per fragment on edge geometry, not face geometry. Edge triangles have wildly different normals than the solid mesh, so the lighting looks completely wrong. Unlit wireframe shows topology without confusing shading artifacts.

What is the halfway vector *H*? Instead of computing the reflection of L and dotting it with V (classic Phong, expensive), Blinn computes $H = \text{normalize}(L + V)$ and measures how closely H aligns with N. When the surface normal bisects L and V, the highlight is at maximum — same visual result, cheaper to compute.

What does Pyodide do here? It runs real CPython inside a WebAssembly sandbox in the browser tab. The Python parser handles OBJ fan-triangulation, and the `calculate_shading` function demonstrates how the $I_l \cdot k_d$ intensity scaling maps an RGB color to its lit counterpart — all without a backend server.

What is the difference between Gouraud and Phong shading? Gouraud shading (used in Lambert) computes lighting once per vertex and interpolates the resulting color across the triangle. Phong shading (used in Blinn-Phong and PBR) interpolates the surface normal across the triangle and evaluates the full lighting equation per fragment, producing significantly more accurate specular highlights at the cost of more shader computation.